

Gardener F5 Application Load Balancer Extension

Part 6 - Runtime Flows and State Machines

Creation, update, scaling, node replacement, recovery, deletion, partial failure, concurrency, sequence diagrams, activity flows, and state models.

Architecture and Low-Level Design

Document role: This part validates the architecture defined in Parts 1-5 by showing how the complete system behaves over time. It treats Kubernetes as the source of truth, keeps controllers thin, rebuilds desired state on every reconcile, and delegates provider convergence to deployment managers and typed CMP clients.

Table of Contents

6.1 Purpose and Runtime Principles
6.2 Runtime Actors and Boundaries
6.3 End-to-End Reconciliation Activity
6.4 Controller State Machine
6.5 Per-Resource Action State Machine
6.6 Creation Flow
6.7 Specification Update Flow
6.8 Backend Scaling Flow
6.9 Node Replacement Flow
6.10 Ingress Rule and TLS Rotation Flow
6.11 Controller Restart and Recovery
6.12 CMP Timeout and Unknown Outcome
6.13 Partial Failure and Resume
6.14 Deletion and Finalizer Flow
6.15 Ownership Conflict During Runtime
6.16 Concurrency and Serialization
6.17 Requeue, Backoff, and Stabilization
6.18 Status and Condition Transitions
6.19 Failure Matrix
6.20 Invariants Across All Flows
6.21 Runtime Pseudocode
6.22 Sequence Diagram Catalogue
6.23 Operational Observability
6.24 Design Decisions
6.25 Implementation Checklist

6.1 Purpose and Runtime Principles

The static component model is useful only if it produces predictable behavior during real lifecycle events. This part specifies that behavior. It defines when the controller reads Kubernetes objects, when desired state is rebuilt, how observed CMP resources are discovered, how actions are ordered, and how the controller behaves when the process or provider fails midway.

- Kubernetes objects and their dependencies are the source of truth.
- Every reconciliation is safe to repeat and does not depend on process memory.
- Desired state is rebuilt before provider mutation.
- Observed state is discovered through typed CMP clients and ownership keys.
- Actions are dependency ordered; deletes are reverse ordered.
- Successful request submission is not equivalent to readiness.
- Partial progress is retained through provider resources and status identifiers, then rediscovered.
- Frontend stability is preferred while backend-only changes are reconciled.
- Finalizers are removed only after owned provider resources are verified absent.

Motive alignment: The flows deliberately follow the AWS Load Balancer Controller architectural pattern - controller, model builder, internal stack, deployer, resource managers - while mapping execution to CMP/F5 resource capabilities. Existing extension code influences transport reuse only, not runtime architecture.

6.2 Runtime Actors and Boundaries

Actor	Runtime responsibility	State it owns
Kubernetes API	Stores Service, Ingress, EndpointSlice, Secret, Node, class, status and finalizer state.	Authoritative user intent and dependency state.
Controller/Reconciler	Loads a coherent snapshot, invokes validation/model building/deployment, writes final outcome.	No provider resource lifecycle state beyond references/status.
Model Builder	Transforms the snapshot into a deterministic desired graph.	Immutable desired objects and logical IDs.
Backend Resolver	Maps ready endpoints to nodes, VMs, network ports and backend_port_id.	Resolution result and explicit unresolved reasons.
Stack Deployer	Discovers observed graph, computes plan, runs actions and aggregates results.	One reconciliation plan and execution result.
Resource Managers	Converge one CMP resource kind.	Kind-specific diff and lifecycle rules.
Typed CMP Clients	Perform typed API operations and poll asynchronous operations.	Transport-level request/response state.
CMP/F5	Stores actual load-balancer resources and executes	Provider-observed state.

	operations.	
Status Writer/Event Recorder	Publishes readiness, VIP, observed generation, errors and warnings.	Kubernetes status and events.

6.3 End-to-End Reconciliation Activity

End-to-End Runtime Reconciliation

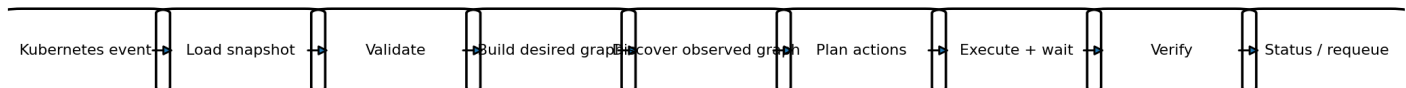


Figure 6-1. End-to-end reconciliation activity.

An event is only a trigger; it is not trusted as a complete description of the change. The reconciler always loads the current objects again. Multiple events may collapse into one queue key, and one event may be obsolete by the time it is processed.

1. Acquire a keyed reconciliation lock for the owner object.
2. Load the latest owner and all required dependencies.
3. Handle deletion before normal desired-state construction.
4. Evaluate controller scope and class ownership.
5. Perform structural, semantic and runtime validation.
6. Resolve backend identities and construct the desired graph.
7. Discover the current CMP graph by stable IDs and ownership keys.
8. Normalize and diff desired versus observed state.
9. Create an action DAG and execute it with bounded concurrency.
10. Wait for required asynchronous operations and re-discover affected resources.
11. Verify the full graph and serving readiness.
12. Write status, conditions and events; remove the finalizer only after verified deletion.

6.4 Controller State Machine

Controller Runtime State Machine

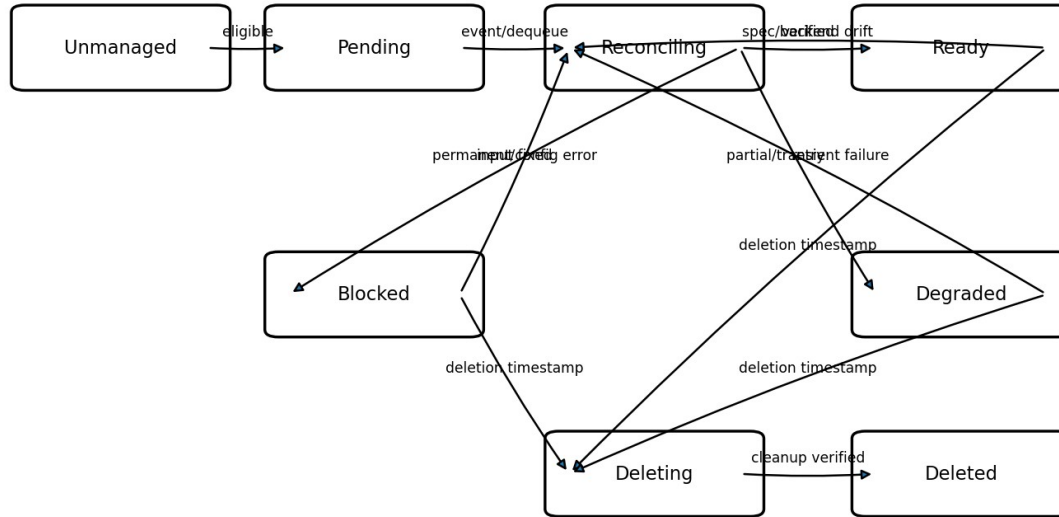


Figure 6-2. High-level owner-object state machine.

State	Meaning	Exit condition
Unmanaged	Object is outside controller scope or handled by another class.	Eligibility/class changes or deletion cleanup is needed.
Pending	Eligible object is queued or waiting for dependencies.	Reconcile starts.
Reconciling	Desired and observed graphs are being converged.	Verified ready, degraded, blocked, or deletion.
Ready	Required frontend and backend graph is verified and serving policy is satisfied.	Spec, dependency, health, drift, or deletion event.
Degraded	A serving graph may exist, but a transient/partial failure prevents full convergence.	Retry succeeds or error becomes permanent.
Blocked	Input, capability, ownership or configuration error cannot be solved by retry alone.	Relevant input changes.
Deleting	Finalizer is retained while owned graph is removed.	All owned resources verified absent.
Deleted	Provider cleanup complete and finalizer removed.	Terminal for that Kubernetes UID.

Condition model: These are logical states, not a new CRD. They are represented through Kubernetes conditions, events, requeue behavior, observedGeneration, and finalizer presence.

6.5 Per-Resource Action State Machine

Per-Resource Action State Machine

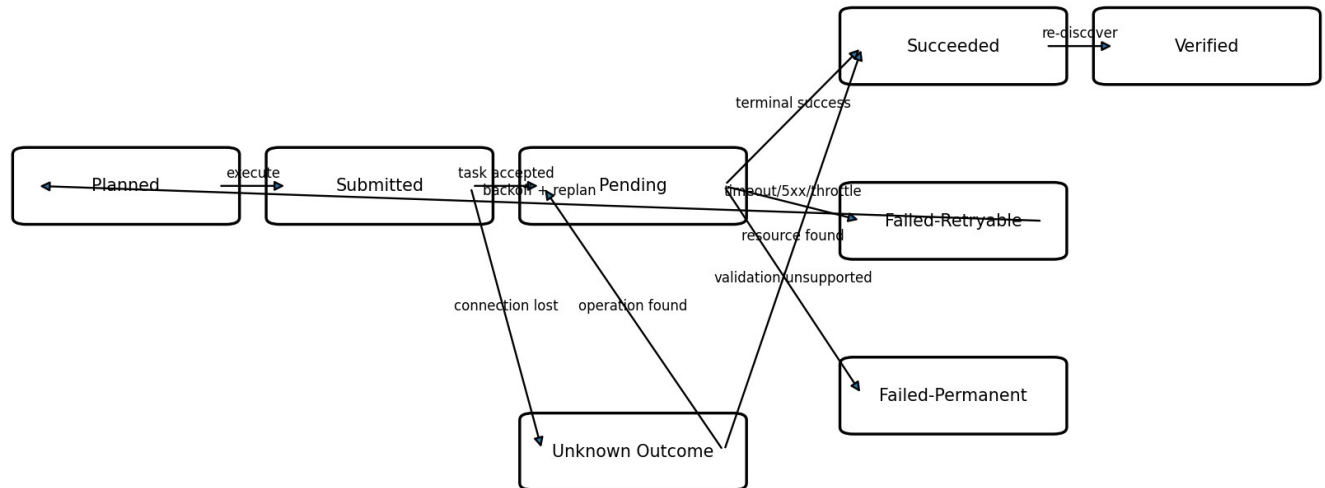


Figure 6-3. State machine for one create, update, or delete action.

The stack may contain many actions, each with its own state. A parent action may block dependent children, while independent branches can continue. Unknown outcomes are never converted directly into another mutation; they first trigger discovery.

- Planned actions contain preconditions, ownership expectations and dependency edges.
- Submitted means the request left the client; Pending means CMP returned an operation/task reference or non-terminal state.
- Succeeded is provisional until resource re-discovery verifies the requested state.
- Retryable failures return to planning after backoff because the observed graph may have changed.
- Permanent failures block only unsafe dependent actions and produce a stable status reason.

6.6 Creation Flow

Creation Sequence

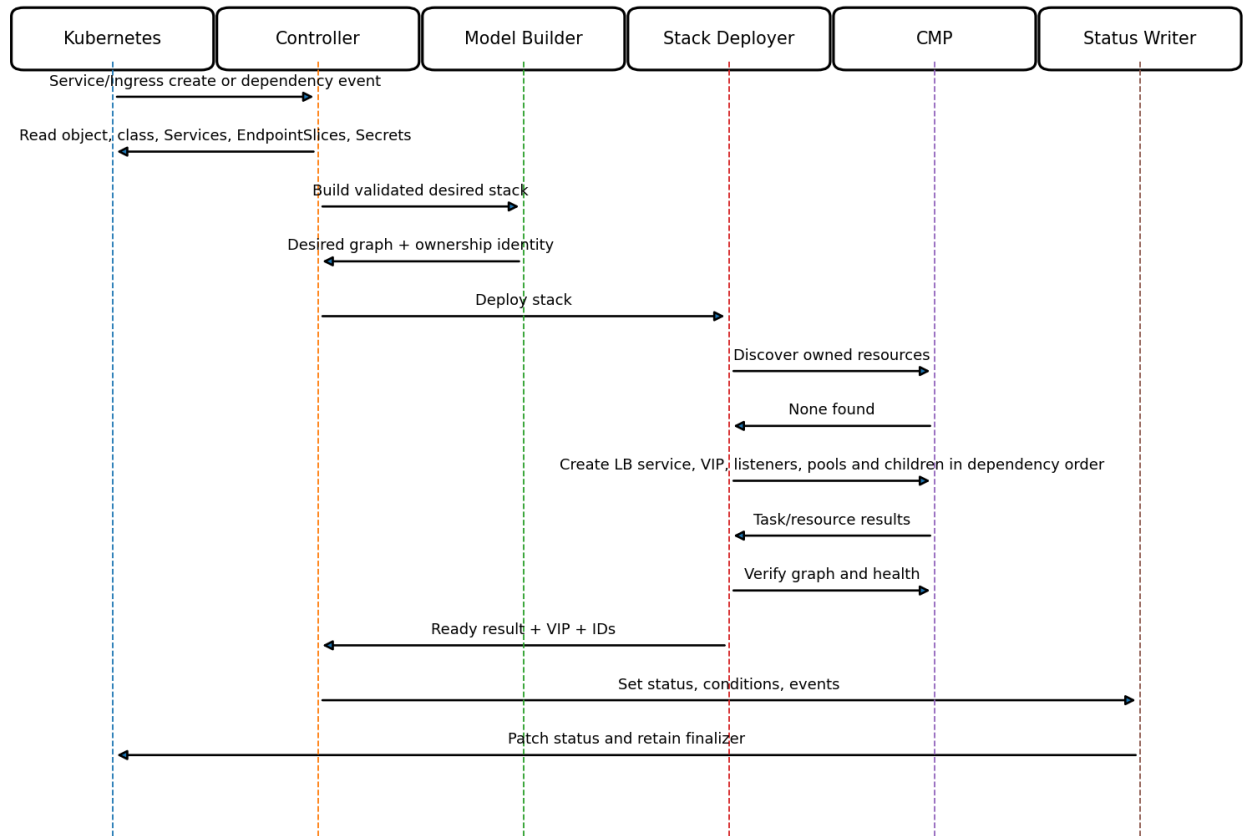


Figure 6-4. New Service or Ingress creation sequence.

6.6.1 Creation Ordering

Order	Resource/action	Readiness gate
1	Add finalizer after eligibility and before first external mutation.	Finalizer patch succeeds.
2	LB Service.	Provider object exists and ownership matches.
3	VIP and address allocation.	Address/port is allocated and stable.
4	Certificates where required.	Certificate object is accepted and usable.
5	Virtual Servers/listeners.	Frontend exists with correct VIP, protocol and port.
6	Pools and monitors.	Each referenced pool exists; monitor association is valid.
7	Members.	Desired member set exists; readiness policy evaluated.
8	Ingress routing rules/default	Every rule target resolves to an

	action.	existing pool.
9	Full graph verification and status.	VIP available and required dependencies verified.

Creation is resumable. If the VIP exists and listener creation fails, the next reconcile adopts the owned VIP and resumes from listener reconciliation; it does not allocate another VIP.

6.7 Specification Update Flow

Specification Update Sequence

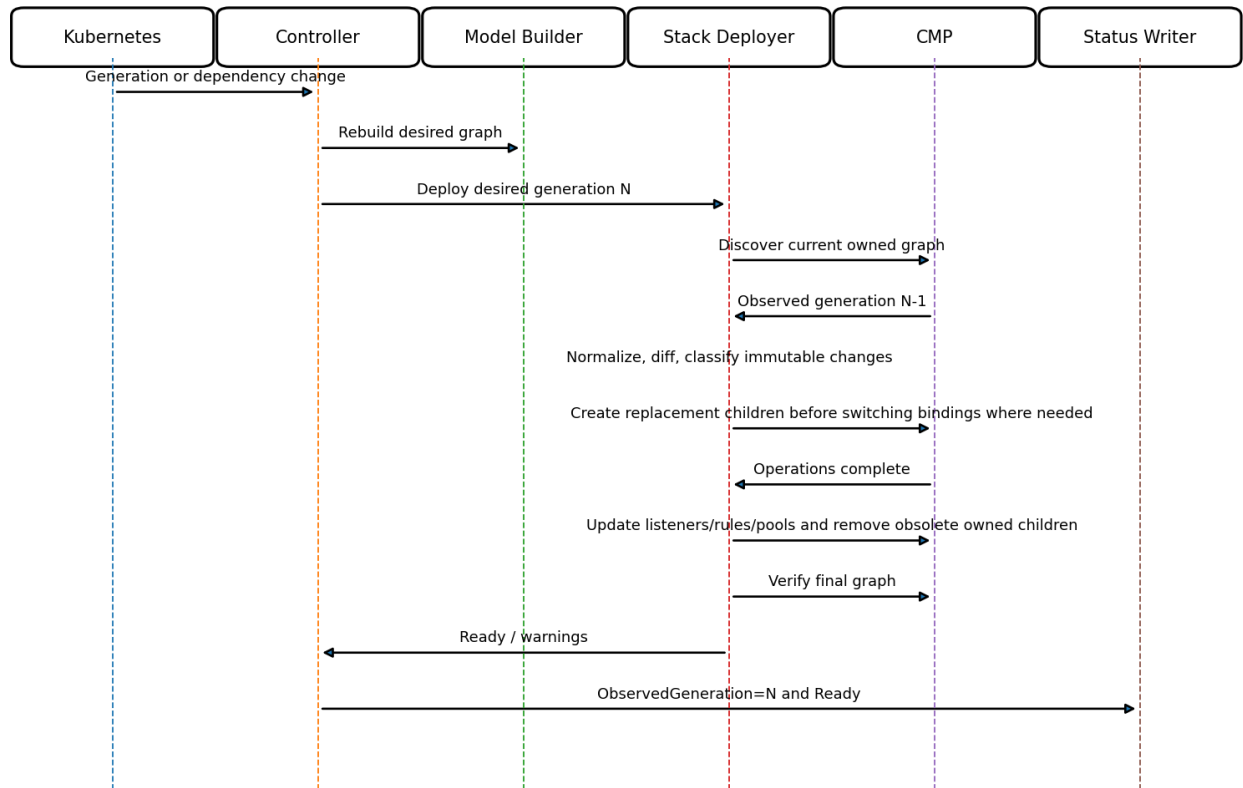


Figure 6-5. Reconciliation after Service, Ingress, annotation, Secret, or class-related update.

6.7.1 Update Classification

Change class	Examples	Runtime behavior
In-place mutable	Pool algorithm, monitor timing, rule priority/action supported by CMP update.	Patch/update resource, wait, verify.
Child replacement	Certificate content, rule representation, monitor kind where API is immutable.	Create replacement, switch binding, remove old child.
Frontend replacement	Protocol/VIP/network property that CMP cannot mutate.	Create replacement frontend under same owner, cut over safely, remove obsolete

		frontend.
Backend-only	Endpoint readiness, node assignment, scale.	Preserve LB service, VIP, listeners and routes; reconcile members.
Unsupported transition	Requested feature cannot be represented safely.	Keep last known serving graph where possible; set Blocked/InvalidConfiguration.

Generation safety: Status.observedGeneration advances only when the desired generation has been successfully evaluated. Ready=True for the new generation is set only after provider verification; an older serving graph must not be presented as successful convergence of the new spec.

6.8 Backend Scaling Flow

Backend Scale Up / Scale Down Sequence

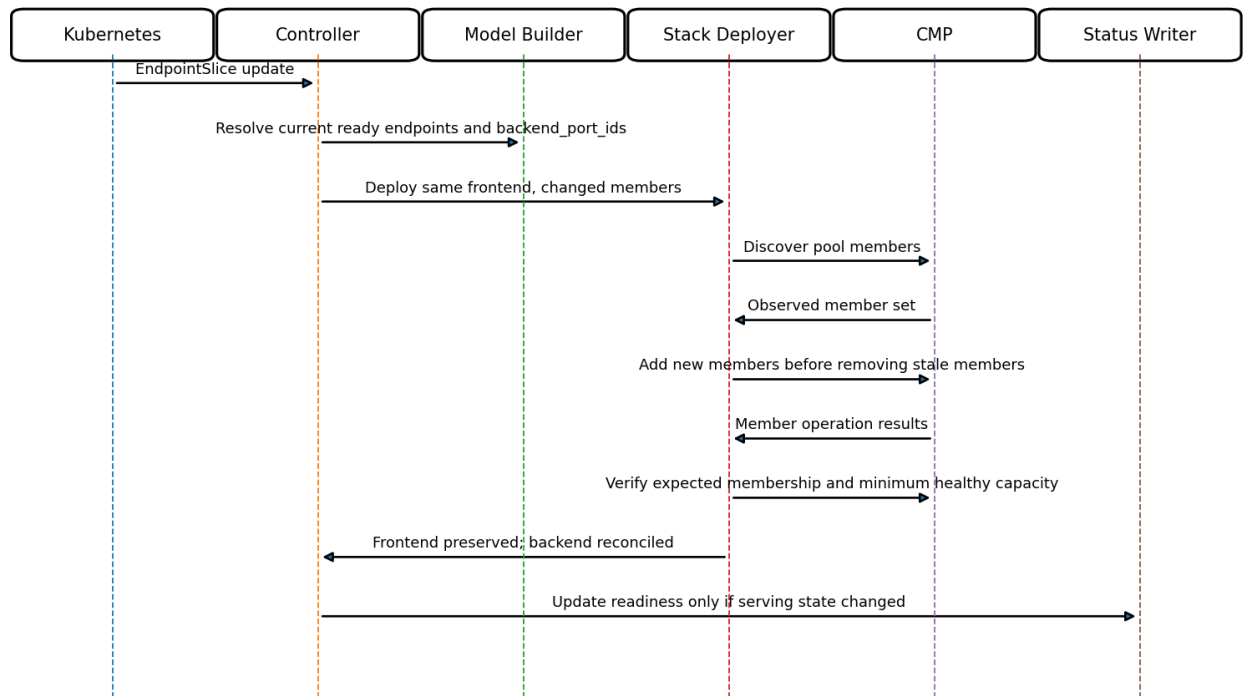


Figure 6-6. Endpoint-driven scale up and scale down.

6.8.1 Scale Up

- Resolve only Ready endpoints unless an explicitly documented policy permits otherwise.
- Add new members without recreating pool or frontend resources.
- Use deterministic member logical IDs based on pool identity and backend identity.
- Verify created members and optionally health before declaring full capacity.

6.8.2 Scale Down

- Compare the complete desired member set with the complete owned observed member set.
- Remove only stale members that are proven owned by the pool/owner.
- Do not delete a pool merely because it temporarily has zero ready endpoints.
- Set a NoReadyBackends/Degraded condition while preserving the frontend for recovery.

6.8.3 Churn Suppression

EndpointSlice updates can arrive in bursts. The queue naturally coalesces keys; an optional short stabilization delay may be used for member-only changes, but it must be bounded and must not delay deletion, security changes, or correction of invalid ownership. The desired set is always recalculated after the delay.

6.9 Node Replacement Flow

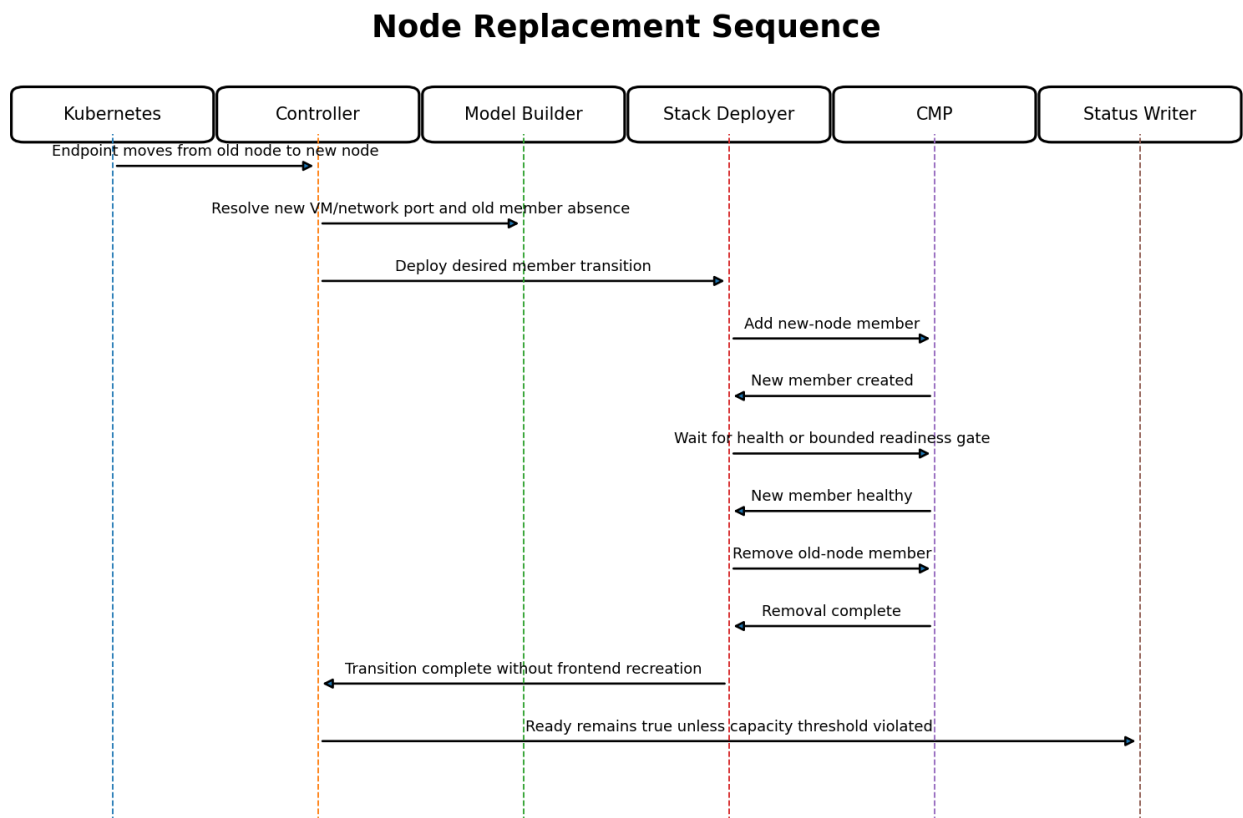


Figure 6-7. Backend transition when a Pod endpoint moves to a replacement node.

Node replacement differs from ordinary scale because the logical application endpoint may remain while the provider identity changes. The resolver produces a new VM/network-port/backend_port_id tuple. Runtime safety uses add-before-remove where capacity and provider behavior allow it.

Phase	Required check	Failure response
Resolve new node	Node provider identity, VM, network and port are	Do not remove old member; requeue with

	unambiguous.	BackendResolutionFailed.
Create new member	New member is owned and points to expected backend_port_id/NodePort.	Retry create/discover unknown outcome.
Readiness gate	New member reaches accepted state or bounded policy threshold.	Keep old member and mark Degraded if timeout.
Remove old member	Old member is absent from desired set and ownership matches.	Retry deletion; do not recreate frontend.
Verify	Exactly expected members remain.	Re-discover and replan.

6.10 Ingress Rule and TLS Rotation Flow

6.10.1 Host/Path Rule Change

13. Build the complete deterministic rule set from the current Ingress group.
14. Create any newly required target pools before rules reference them.
15. Apply rules in deterministic priority order.
16. Verify that no rule points to a missing or foreign pool.
17. Remove obsolete rules and then unreferenced owned pools.

6.10.2 TLS Secret Rotation

18. Secret event enqueues every owning/grouped Ingress that references it.
19. Read and validate the new certificate/key pair.
20. Compute a content fingerprint and desired certificate logical identity.
21. Create or update the CMP certificate according to API mutability.
22. Bind the verified new certificate to HTTPS virtual servers.
23. Only after binding verification, delete an obsolete certificate that has no remaining owned references.

No plaintext persistence: Private key material is held only for the minimum request lifetime, is never placed in status/events/logs, and is not stored in the internal model beyond the secure deployment boundary.

6.11 Controller Restart and Recovery

Controller Restart and Unknown-Outcome Recovery

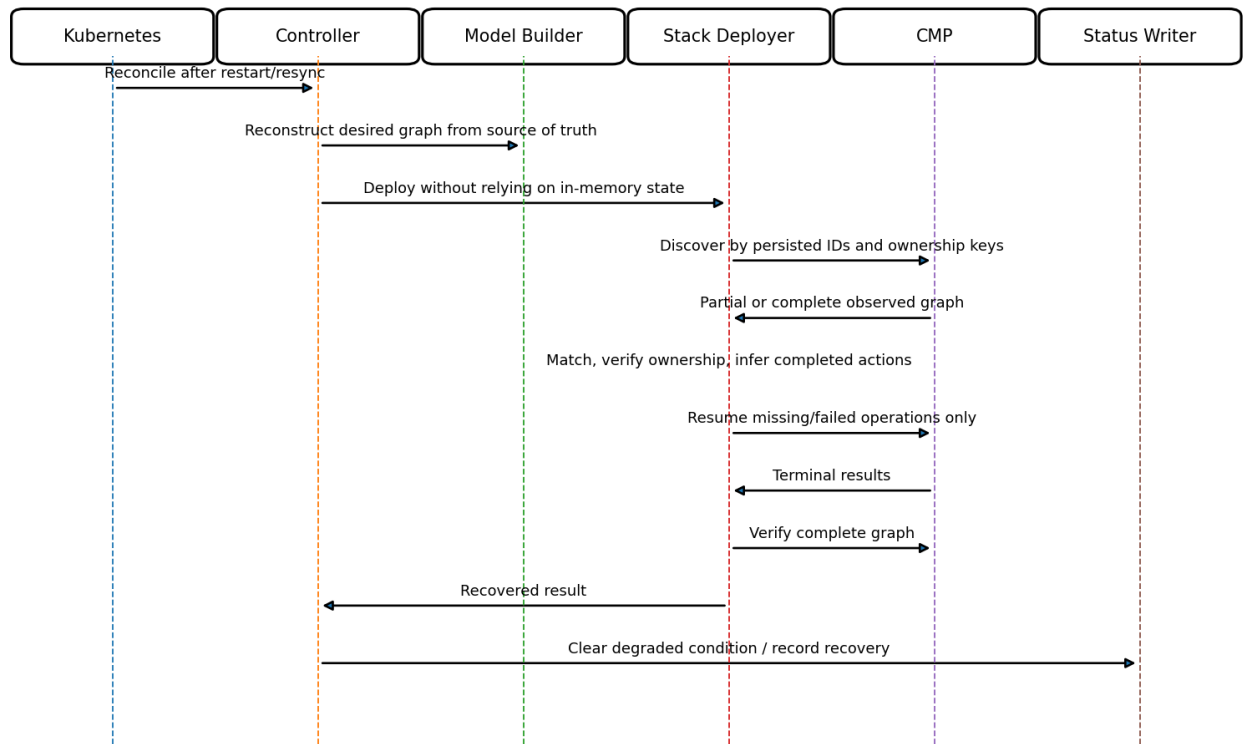


Figure 6-8. Restart-safe recovery by reconstruction and discovery.

The controller must behave correctly if it stops after any successful CMP mutation but before status is written. Recovery therefore cannot rely on in-memory operation records. Persisted CMP IDs are useful accelerators, but ownership-based discovery is the safety mechanism.

- Rebuild desired state from current Kubernetes objects.
- Use status IDs when present, but verify the returned resource ownership and type.
- Fall back to ownership lookup when status is missing, stale, or points to a deleted object.
- Treat an owned matching resource as completed work, even if the previous request response was lost.
- Recalculate the entire action plan; never blindly continue an old serialized plan.
- Repair status after provider convergence is verified.

6.12 CMP Timeout and Unknown Outcome

A timeout does not prove failure. The request may have been accepted and the response lost. Retrying the same create without discovery can create duplicates.

24. Classify whether the timeout occurred before request transmission, during response wait, or while polling an accepted task.
25. When an idempotency key is supported, reuse the same key for the same logical mutation.

26. Discover by returned resource reference, task ID, deterministic ownership key and logical name.
27. If exactly one owned matching resource is found, continue with verification.
28. If none is found after an observation window, retry according to policy.
29. If multiple candidates are found, stop mutation and raise OwnershipConflict/AmbiguousProviderState.

Outcome	Controller action
Task pending	Requeue after server hint or bounded poll interval.
Task succeeded, resource visible	Verify fields and continue dependencies.
Task succeeded, resource not yet visible	Short consistency requeue; do not resubmit immediately.
Connection lost, matching resource found	Adopt as completion of the intended action.
Connection lost, no resource found	Retry after backoff/observation deadline.
Multiple matching resources	Block and require safe operator/provider cleanup decision.

6.13 Partial Failure and Resume

Partial Failure: Persist, Re-discover, and Resume

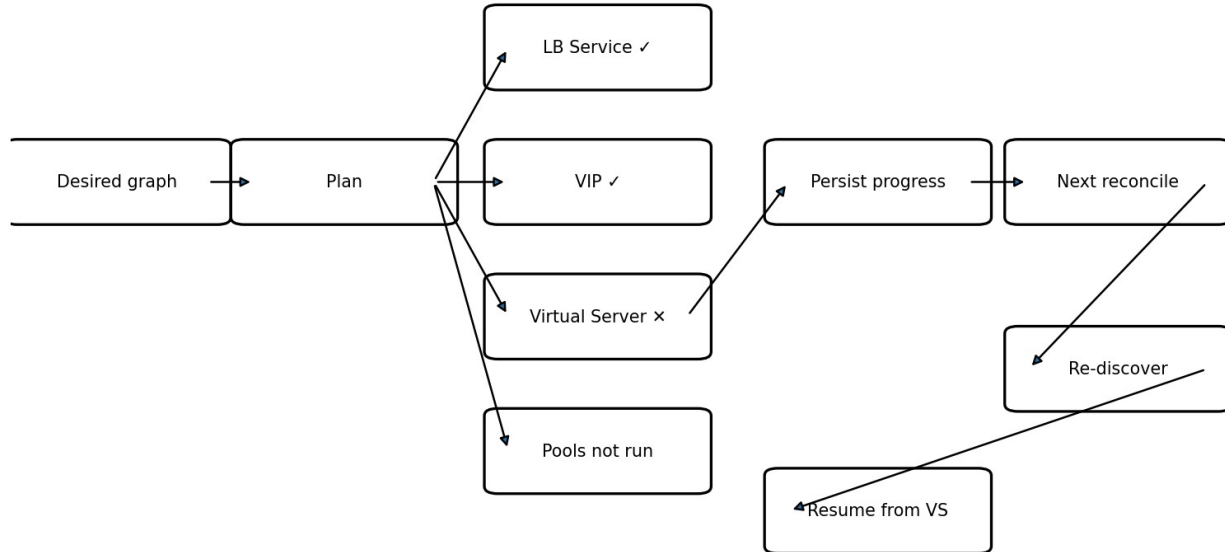


Figure 6-9. Partial graph failure is resumed, not rolled back blindly.

6.13.1 Failure Containment

- Stop actions whose prerequisites failed.
- Allow independent branches to complete only when doing so cannot expose invalid traffic.

- Record successful CMP IDs and operation references in the aggregate result where status schema permits.
- Emit one concise owner-level event plus structured logs for individual action failures.
- Prefer convergence on the next reconcile over immediate destructive rollback.

6.13.2 Compensation Policy

Compensating deletion is used only when the newly created resource is known to be unused, owned, and harmful to leave behind. It is not used for every failure because rollback can itself fail and can destroy a previously serving graph. Examples of safe compensation include an unbound replacement certificate or an empty replacement pool created during a failed cutover.

6.14 Deletion and Finalizer Flow

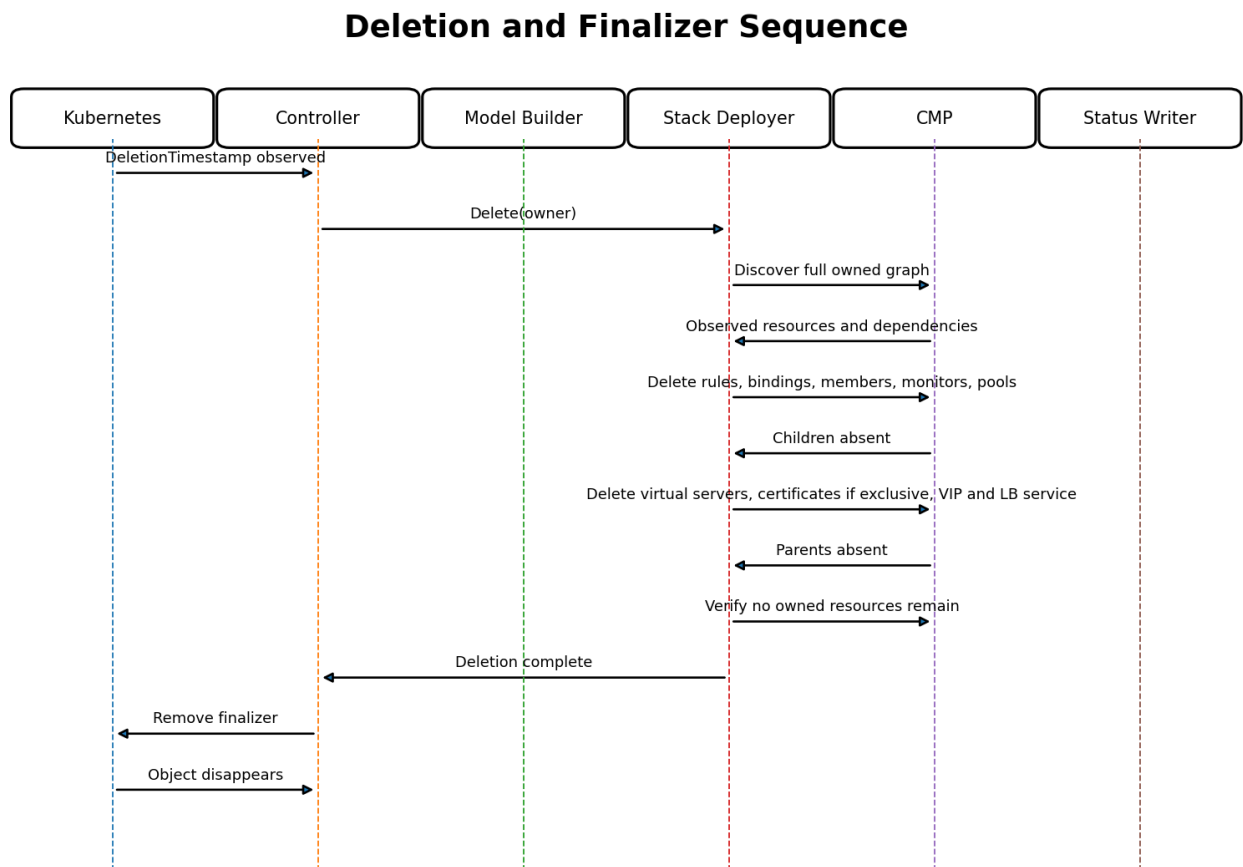


Figure 6-10. Owner deletion and finalizer sequence.

6.14.1 Reverse Dependency Order

30. Stop normal create/update reconciliation once DeletionTimestamp is observed.
31. Discover the full owned graph, including resources missing from Kubernetes status.
32. Delete routing rules and certificate/listener bindings.
33. Delete members and monitors, then pools.
34. Delete virtual servers/listeners.

35. Delete certificates only when exclusively owned and unreferenced.
36. Delete VIP and finally the LB Service.
37. Re-discover and verify that no owned resources remain.
38. Remove the Kubernetes finalizer.

6.14.2 Deletion Safety

- Never delete by display name alone.
- Never delete a resource whose ownership is absent, conflicting, or ambiguous.
- A NotFound response is successful only for the exact expected resource scope.
- Shared/group resources use reference-aware ownership; one member leaving a group does not delete the group while others remain.
- If credentials are invalid, retain the finalizer and publish a clear DeletionBlocked condition rather than leaking resources silently.

6.15 Ownership Conflict During Runtime

Situation	Decision	Status
Expected ID resolves to foreign owner	Do not update/delete; attempt ownership-key discovery for correct resource.	OwnershipConflict.
Owned resource name collides with foreign resource	Generate/use deterministic collision-resistant provider name if create is safe; otherwise block.	ProviderNameConflict.
Resource has legacy metadata but uniquely matches explicit adoption policy	Adopt only when policy is enabled and all immutable attributes match.	Adopted with warning/event.
Multiple legacy candidates match	No adoption or mutation.	AmbiguousProviderState.
Ownership metadata removed externally	Treat as drift and unsafe; do not restore metadata without verified identity policy.	OwnershipLost.
Resource moved under another parent	Block delete/update until graph identity is resolved.	OwnershipGraphConflict.

6.16 Concurrency and Serialization

Concurrency and Serialization Boundaries

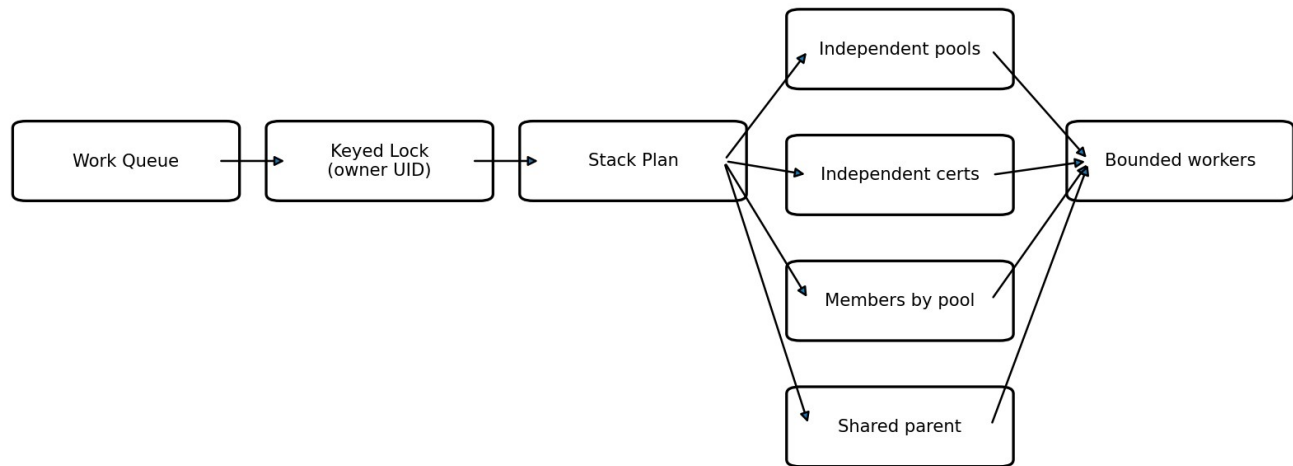


Figure 6-11. Runtime concurrency boundaries.

The controller may reconcile different owner UIDs concurrently. Reconciliation for the same owner is serialized. Within one action plan, independent resource branches may execute concurrently under a small bounded worker pool; shared parents and ordered cutovers remain serialized.

- Key locks use cluster identity plus Kubernetes UID, not namespace/name alone.
- Ingress-group reconciliation uses the group ownership key so two members cannot mutate the shared graph concurrently.
- CMP client rate limiting is global per credential/endpoint and honors Retry-After where available.
- Member actions may be batched by pool, but batches are isolated so one pool failure does not corrupt another.
- Cancellation stops submission of new actions but does not assume already submitted CMP operations were cancelled.

6.17 Requeue, Backoff, and Stabilization

Reason	Typical timing strategy	Reset condition
Asynchronous task pending	Server-provided retry hint or short bounded interval.	Terminal task/resource state.
Transient network/5xx	Exponential backoff with jitter and maximum bound.	Successful CMP interaction.
Rate limited	Retry-After or provider quota window.	Successful request / quota recovery.
Dependency not ready	Moderate requeue plus	Dependency event or successful

	dependency watch.	resolution.
No ready endpoints	Longer bounded requeue plus EndpointSlice watch.	Ready endpoint appears.
Permanent validation/capability error	No periodic hot retry; rely on relevant object change, optional slow safety resync.	Generation/dependency change.
Deletion blocked by transient provider failure	Backoff, but retain finalizer.	Cleanup succeeds.
Event burst/member churn	Optional small debounce/stabilization.	Latest snapshot processed.

Backoff state is queue-level and disposable. Correctness never depends on retaining it across restarts.

6.18 Status and Condition Transitions

Condition	True when	False/cleared when
Accepted	Class, structure, semantics and CMP capability are valid.	Validation or ownership policy fails.
BackendResolved	Every required backend can be mapped to CMP identity.	Any required backend is unresolved/ambiguous.
Progressing	At least one required provider action is pending.	Terminal ready, blocked, or deletion result.
Ready	Required graph is verified and serving threshold is met.	New generation not converged, no backends, or critical drift.
Degraded	A graph exists but transient/partial failure or reduced capacity remains.	Full convergence restored.
OwnershipConflict	Observed provider resource cannot be safely associated.	Conflict is resolved and graph verified.
DeletionBlocked	Finalizer cleanup cannot proceed safely.	All owned resources removed.
Unsupported	Requested behavior has no accepted CMP mapping.	Spec changes to supported configuration.

6.18.1 Status Write Rules

- Use patch with conflict retry and avoid rewriting unchanged conditions.
- Preserve lastTransitionTime when only message/detail changes without condition status transition.
- Include observedGeneration for conditions tied to owner spec.
- Do not publish a VIP until allocation is verified and ownership matches.
- Do not clear a previously published VIP during a temporary CMP read failure unless deletion or verified replacement requires it.
- Messages are concise and user-facing; detailed request IDs remain in logs/metrics.

6.19 Failure Matrix

Failure point	Serving impact	Automatic action	Condition / event
Validation before mutation	None; old graph preserved.	Wait for input change.	Accepted=False / InvalidConfiguration.
Backend resolution	Frontend may remain; capacity stale.	Retry on Node/Endpoint/provider inventory change.	BackendResolved=False.
LB Service create	No new frontend.	Discover unknown outcome, then retry.	Progressing/Degraded.
VIP allocation	No externally usable endpoint.	Poll/retry; preserve owned parent.	VIPPending.
Listener/VS create	VIP may exist but not serve.	Resume at listener.	Ready=False.
Pool/member update	Frontend may serve reduced/stale capacity.	Reconcile failed branch; preserve parent.	Degraded=True.
Rule update	Some routes may remain old.	Use deterministic full-set replan; avoid deleting target pools early.	RoutingUpdateFailed.
Certificate rotation	Old certificate remains bound where safe.	Retry replacement/binding; delete only unused new object if compensating.	TLSUpdateFailed.
Status patch conflict	Provider graph may already be ready.	Reload and patch status only; no provider mutation required.	Usually no warning event.
Deletion provider timeout	Resource may or may not be gone.	Discover and continue cleanup.	DeletionBlocked/Progressing.
Foreign ownership discovered	Unsafe mutation prevented.	Stop and require resolution.	OwnershipConflict=True.

6.20 Invariants Across All Flows

- A controller never calls raw CMP transport directly.
- A model builder never mutates CMP or Kubernetes status.
- No resource is updated or deleted before ownership verification.
- No child is created before required parent identifiers are verified.
- Deletion order is the reverse of dependency order.
- Every retry starts with fresh Kubernetes input and provider discovery.
- An unknown request outcome causes discovery, not blind duplicate mutation.
- Backend-only changes do not recreate VIPs, LB Services, listeners, certificates, or routing rules.
- Status Ready=True implies verified provider state, not only successful API submission.
- Finalizer removal implies no owned external resources remain.
- Logical IDs remain stable for semantically unchanged resources.
- Unsupported behavior fails explicitly rather than being approximated silently.

6.21 Runtime Pseudocode

```

func (r *Reconciler) Reconcile(ctx context.Context, key NamespacedName) (Result, error) {
    owner, err := r.loadOwner(ctx, key)
    if IsNotFound(err) { return Done(), nil }

    if owner.DeletionTimestamp != nil {
        result, err := r.deployer.Delete(ctx, ownership.From(owner))
        return r.finishDeletion(ctx, owner, result, err)
    }

    if !r.scope.Manages(owner) { return Done(), nil }
    if err := r.ensureFinalizer(ctx, owner); err != nil { return Retry(err) }

    snapshot, err := r.loader.LoadSnapshot(ctx, owner)
    if err != nil { return r.status.FromLoadError(ctx, owner, err) }

    validation := r.validator.Validate(snapshot)
    if !validation.Accepted { return r.status.Blocked(ctx, owner, validation) }

    desired, err := r.builder.Build(ctx, snapshot)
    if err != nil { return r.status.FromBuildError(ctx, owner, err) }

    deployed, err := r.deployer.Deploy(ctx, desired)
    return r.status.FromDeployment(ctx, owner, deployed, err)
}

func (d *StackDeployer) Deploy(ctx context.Context, desired *model.Stack) (*Result, error) {
    observed, err := d.discovery.LoadOwnedGraph(ctx, desired.Owner)
    if err != nil { return nil, classify(err) }

    plan, err := d.planner.Build(desired, observed)
    if err != nil { return nil, err }

    progress := d.executor.Run(ctx, plan) // dependency-aware, bounded concurrency
    verified, verifyErr := d.verifier.Verify(ctx, desired.Owner, desired)
    return Aggregate(progress, verified), Combine(progress.Err, verifyErr)
}

```

6.22 Sequence Diagram Catalogue

The following diagrams are normative runtime references. Each is tied to the common state and ownership rules above rather than defining a separate implementation path.

Diagram	Primary trigger	Key architectural property
Creation	New eligible Service/Ingress or newly formed group.	Desired model before mutation; dependency ordering; verified status.
Update	Owner generation, annotation, dependency, rule or Secret change.	Full re-diff; replacement only where necessary.
Scale	EndpointSlice membership/readiness change.	Backend-only convergence and frontend preservation.
Node replacement	Endpoint moves to new node/provider identity.	Add-before-remove and readiness gate.
Recovery	Controller restart, resync, lost response.	Reconstruction plus ownership discovery.
Deletion	DeletionTimestamp/finalizer.	Reverse dependency cleanup and verified absence.

6.23 Operational Observability

6.23.1 Structured Logs

- owner kind/namespace/name/UID and cluster identity
- reconcile ID, desired generation and stack hash
- logical resource ID, CMP resource ID and operation/task ID
- action type, attempt, duration and classified outcome
- ownership/adoption decision without credentials or certificate material

6.23.2 Metrics

- reconcile duration and result by owner kind/reason
- CMP request latency/error code/operation state by endpoint family
- action counts by create/update/delete/no-op
- pending asynchronous operation count and age
- ownership conflicts and ambiguous-outcome recoveries
- backend resolution failures by stage
- finalizer deletion duration and blocked deletions
- ready/degraded object gauges where cardinality controls permit

6.23.3 Events

Events are rate limited and owner-focused. Emit for accepted/rejected configuration, VIP allocation, successful readiness, ownership conflict, repeated provider failure, TLS failure, and deletion blockage. Do not emit an event for every poll or every individual member during normal scaling.

6.24 Design Decisions

Decision	Rationale
Reconstruct desired state every reconcile	Makes restart, event coalescing and stale queue entries safe.
Discover before every mutation plan	Handles external drift, lost responses and partial prior progress.
Logical state machines, not workflow persistence CRDs	Keeps architecture aligned with controller-runtime convergence and avoids a second source of truth.
Preserve frontend during backend churn	VIP/listener stability is operationally critical and backend changes are frequent.
Add-before-remove for replacements	Reduces avoidable traffic loss when capacity and CMP semantics permit.
No blind rollback	A rollback can fail or destroy a serving graph; next-reconcile convergence is safer.
Explicit blocked state for permanent errors	Prevents hot retry loops and provides actionable user feedback.
Reverse-order verified deletion	Protects dependencies and prevents finalizer removal with leaked resources.
Serialize by owner/group key	Prevents concurrent plans from racing on the same CMP graph.

Status based on verification	HTTP success or task submission alone does not prove serving readiness.
------------------------------	---

6.25 Implementation Checklist

- Controller watches owner and all dependency objects with deterministic enqueue mapping.
- Keyed serialization exists for Service UID, standalone Ingress UID and Ingress group identity.
- Deletion path executes before normal build/deploy path.
- Every normal mutation is protected by a finalizer.
- Desired model and logical IDs are deterministic across restarts.
- Stack deployer discovers complete owned graph before planning.
- Executor supports dependency edges, bounded concurrency, cancellation and partial result aggregation.
- Typed CMP clients expose operation/task polling and classify ambiguous outcomes.
- Backend member reconciliation is isolated from frontend resources.
- Node replacement uses add-before-remove when safe.
- TLS rotation binds verified replacement before deleting old certificate.
- Status conditions implement Accepted, BackendResolved, Progressing, Ready, Degraded and deletion/ownership errors.
- Retries use jittered bounds and honor provider hints.
- Metrics/logs/events carry reconcile and provider correlation IDs without secrets.
- Tests cover failure after every dependency layer, controller restart, duplicate event, stale status ID, foreign resource, timeouts and deletion retry.

Completion criterion: Part 6 is implemented when all lifecycle tests can stop the controller or inject a CMP failure after any action, restart reconciliation, and still converge to the exact desired owned graph without duplicating or deleting foreign resources.

Part 6 Runtime Reference Summary

Runtime event	Resources expected to remain stable	Resources expected to change
Initial creation	None.	Entire owned graph.
Service backend scale	LB Service, VIP, VS, pool configuration, monitors.	Pool members only.
Ingress path/host edit	LB Service, VIP, listener, unaffected pools/certs.	Rules and newly required/obsolete pools.
TLS rotation	LB Service, VIP, routes, pools, members.	Certificate and listener binding.
Node replacement	Frontend and pool identity.	Member provider identity/backend_port_id.
Controller restart	All correctly owned provider resources.	Only missing/drifted resources and repaired status.
Deletion	Nothing after completion.	All owned resources in reverse order.

This reference closes the runtime-design layer. Part 7 can now define package boundaries, interfaces, tests, and the implementation roadmap against these normative flows.